TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Wilson Ignacio Intriago Romero, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Gema Alexandra Reyna Vega

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 202

Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

1. ANÁLISIS DEL PROBLEMA

Los refugios de animales necesitan organizar la información sobre los rescates, la atención veterinaria y las adopciones. La falta de un sistema centralizado puede dificultar el seguimiento de los animales y la coordinación del personal.
Por ello, se propone diseñar un Sistema de Control de Rescate y Adopción Animal que permita registrar perros, gatos y fauna silvestre, asignar voluntarios o veterinarios, actualizar el estado de cada caso y enviar notificaciones al personal encargado. Para su diseño se aplicarán patrones de software, principios SOLID y Gitflow mediante GitHub.

1.1. Identificación de los actores principales

Ciudadano o reportante: informa sobre animales que necesitan rescate y proporciona los datos del caso.

Voluntario: colabora en el rescate y realiza las tareas que le son asignadas.

Veterinario: evalúa la salud de los animales y registra sus tratamientos.

Administrador del refugio: registra y supervisa los casos, asigna responsables y controla los estados de los animales.

1.2. Requerimientos funcionales

Son las funciones que debe realizar el sistema.

- **RF01.** Registrar nuevos casos de rescate.
- **RF02.** Clasificar los animales en perros, gatos y fauna silvestre.
- **RF03.** Asignar voluntarios y veterinarios responsables.
- **RF04.** Actualizar los estados: Rescatado, En tratamiento y En adopción.
- **RF05.** Enviar notificaciones al personal encargado.
- **RF06.** Consultar la información y el historial de cada caso.

1.3. Requerimientos no funcionales

Son las características de calidad que debe cumplir el sistema.

- **RNF01. Escalabilidad:** permitir incorporar nuevas funciones y especies.
- **RNF02. Mantenibilidad:** facilitar las modificaciones y mejoras del sistema.
- **RNF03. Seguridad:** proteger la información y controlar el acceso de los usuarios.
- **RNF04. Trazabilidad:** registrar los cambios del proyecto mediante Git y GitHub.

2. Modelado de Interacciones y Estructura Lógica

2.1 Diagrama de casos de uso

El diagrama de casos de uso representa las interacciones entre los actores principales y sistema de control de rescate y adopción animal.

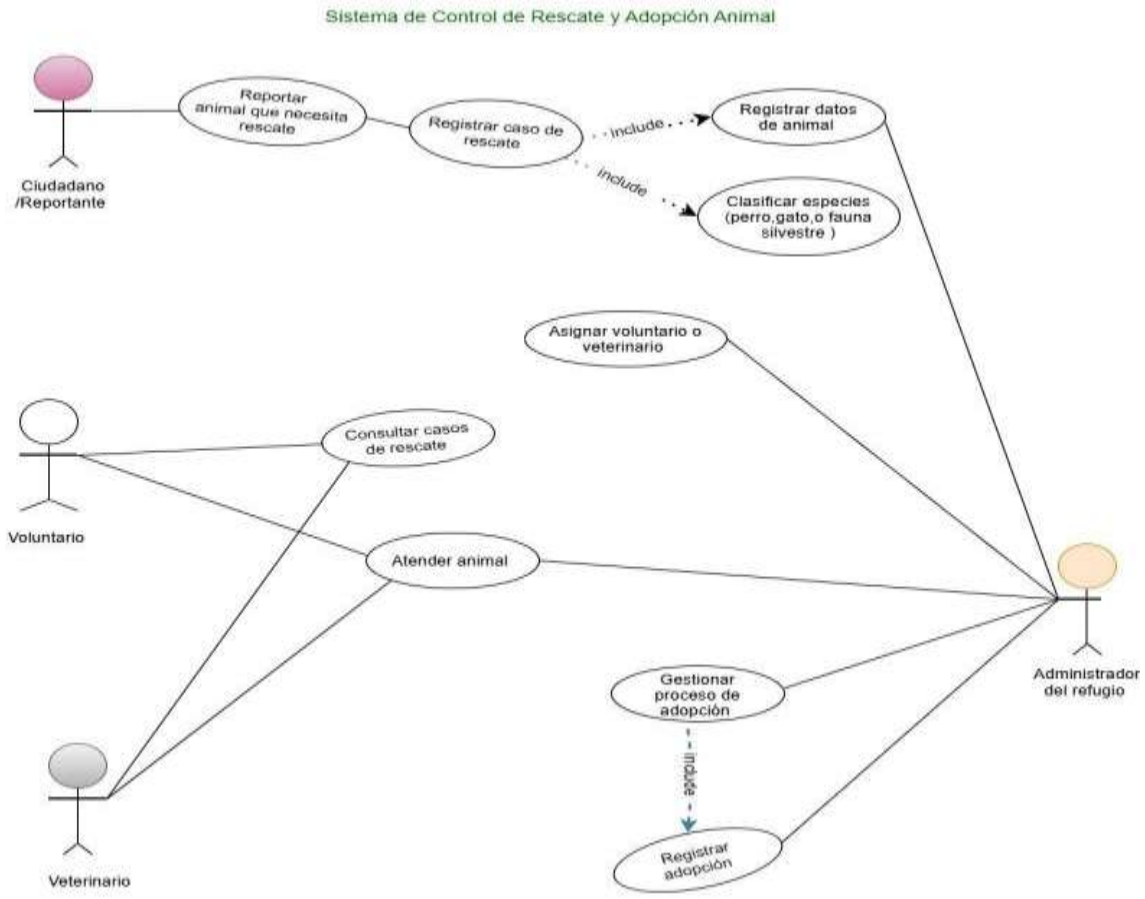


Fig.1: Diagrama de casos de uso

2.2. Diagrama de clases UML

El diagrama de clases representa la estructura estática del Sistema de Control de Rescate y Adopción Animal. Identifica las clases responsables de registrar los casos de rescate, almacenar la información de los animales, asignar voluntarios y veterinarios, actualizar los estados y gestionar las adopciones. También contempla un componente de notificaciones para comunicar los eventos relevantes al personal encargado. La organización propuesta busca facilitar el mantenimiento, la ampliación y la trazabilidad del sistema

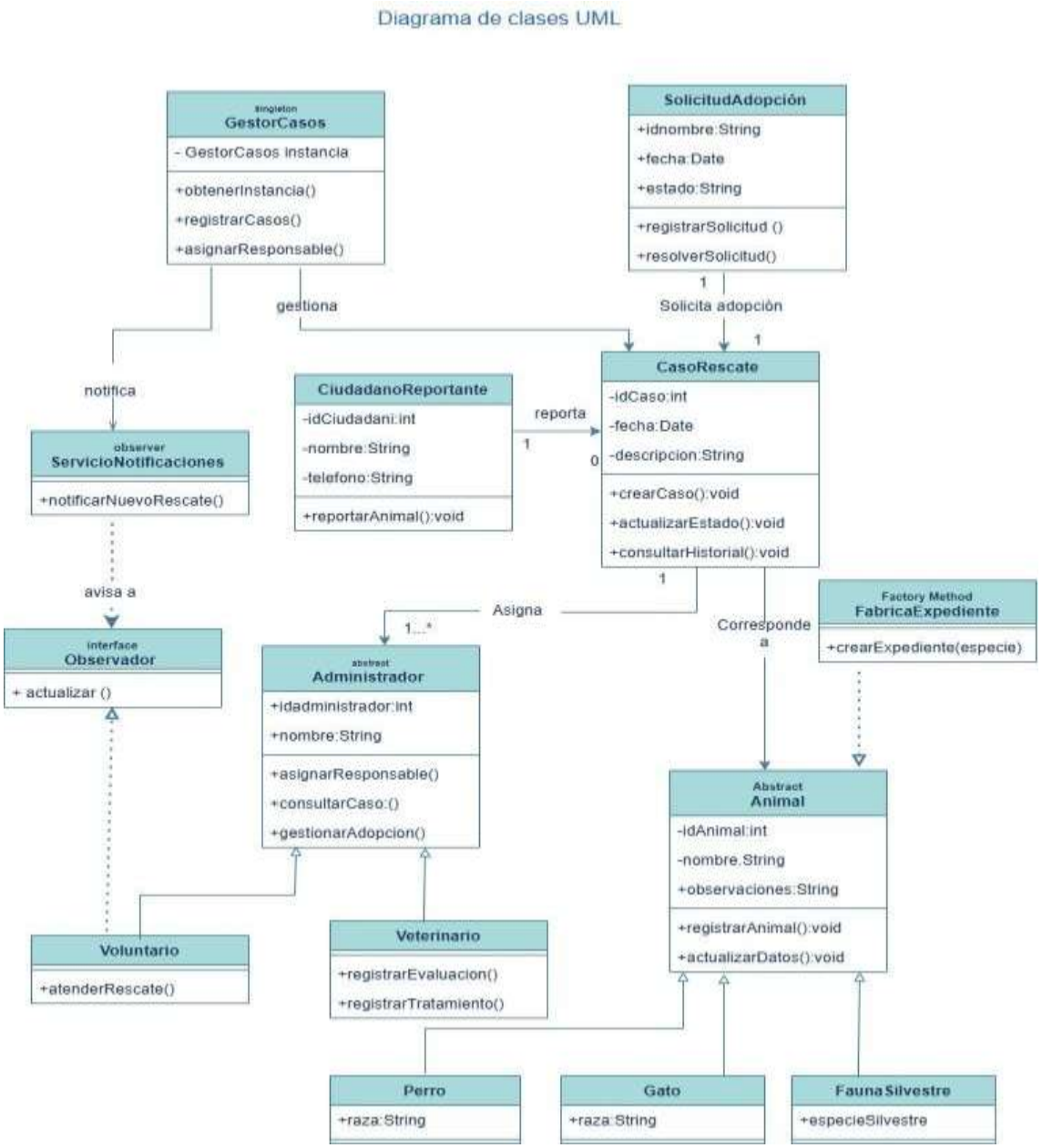
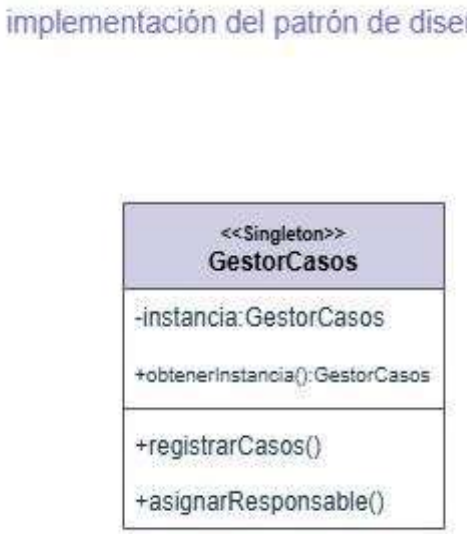


Fig.2: Diagrama de clases UML

3. Aplicación de Patrones de Diseño

3.1. Implementación del Patrón de Diseño: Singleton

El patró**n** Singleton es un patrón de diseño creacional cuyo propósito principal en este sistema es garantizar que la clase GestorCasos tenga una única instancia activa en memoria durante todo el ciclo de vida de la aplicación. Además, provee un punto de acceso global a dicha instancia para todos los demás componentes o actores que requieran interactuar con el control de rescates.



Se aplica en este motor central para evitar que los reportes de rescate se dupliquen en memoria y que los estados de los animales se desincronicen. En el diagrama UML, esto se sostiene mediante el atributo privado -instancia y el método de acceso público +obtenerInstancia()

Fig.3: Implementación del patrón de diseño: Singleton

3.2. Implementación del Patrón de Diseño: Observer

Automatiza el envío de alertas en tiempo real a los voluntarios cuando se registra un nuevo rescate.

- **Funcionamiento:** La clase ServicioNotificaciones actúa como el emisor central; cuando ocurre un ingreso, invoca el método +actualizar () de la interfaz Observador para notificar instantáneamente a la clase Voluntario.
- **Beneficio técnico:** Permite que el sistema sea reactivo y desacoplado, evitando que el emisor de la alerta dependa rígidamente del código de los usuarios que la reciben.



Fig.4: Implementación del Patrón de Diseño: Observer

3.3: Implementación del Patrón Factory Method

Centraliza y automatiza la creación de expedientes de animales por especie.

- **Funcionamiento:** La clase FabricaExpediente se encarga de instanciar dinámicamente los objetos específicos (Perro, Gato o Fauna Silvestre) delegando la lógica de creación fuera del motor principal.
- **Beneficio técnico:** Aplica el principio de Responsabilidad Única de SOLID, haciendo que el sistema sea escalable para añadir nuevas especies en el futuro sin modificar el código existente.

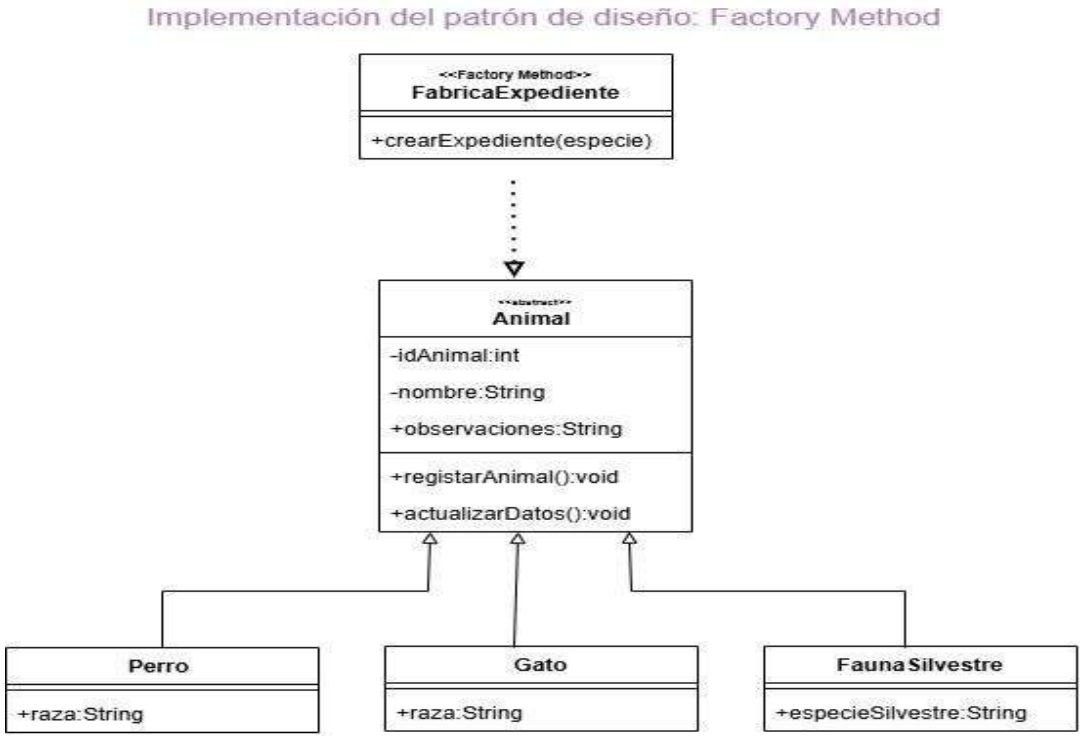


Fig.5: Implementación del Patrón Factory Method

4. Arquitectura General del Sistema

Vista: Es la interfaz con la que interactúan los actores (ciudadanos y personal) para registrar reportes y visualizar las alertas de rescate.

Controlador (GestorCasos): Actúa como el cerebro centralizado bajo el patrón Singleton; recibe las peticiones de la interfaz y coordina todas las acciones lógicas.

Modelo: Contiene las reglas del negocio administradas por los patrones de diseño (creación de especies con Factory Method y despacho de alertas reactivas con Observer), guardando finalmente la información en la base de datos.

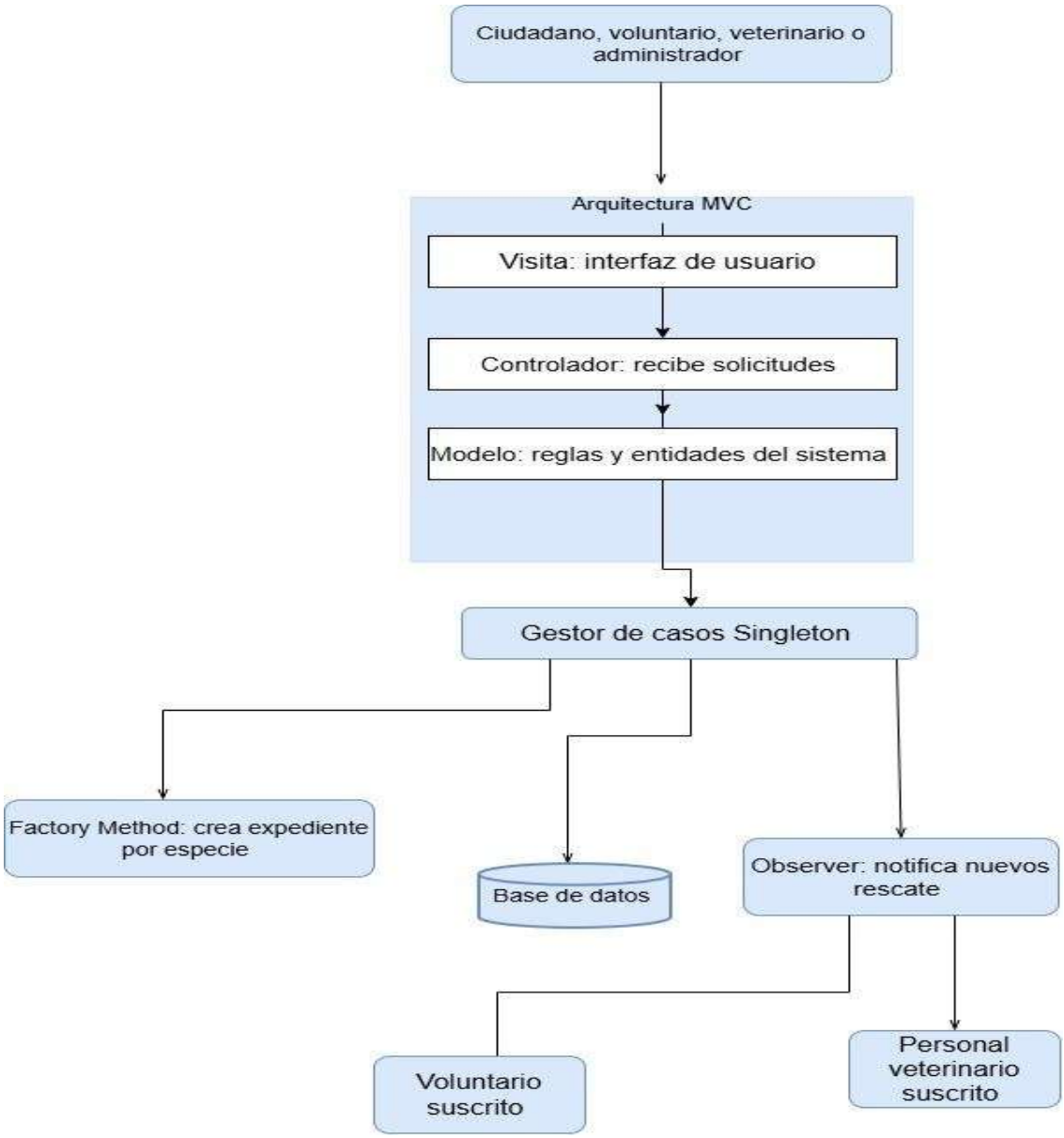


Fig.6: Arquitectura General del Sistema

5.1. Diagrama de Secuencia: Inicialización del Motor (Patrón Singleton)

Se Muestra el flujo temporal donde el Administrador interactúa con el motor único del sistema para registrar un caso.

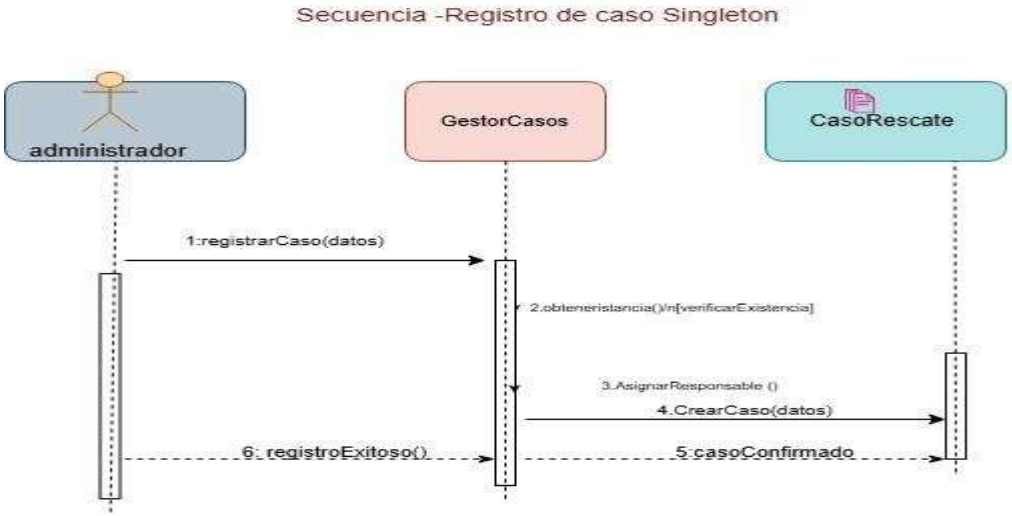


Fig.6: Inicialización del Motor (Patrón Singleton)

El actor invoca 1. registrarCaso(). La clase GestorCasos ejecuta internamente 2. obtenerInstancia() para garantizar el acceso al objeto único en memoria y luego procesa la operación 3. asignarResponsable(). Finalmente, delega la persistencia de la información a la entidad CasoRescate, la cual retorna una confirmación de éxito.

5.2. Diagrama de Secuencia: Sistema de Alertas Automáticas Patrón Observer e Integración Singleton

Modela el comportamiento reactivo del sistema cuando ingresa un nuevo animal al refugio.

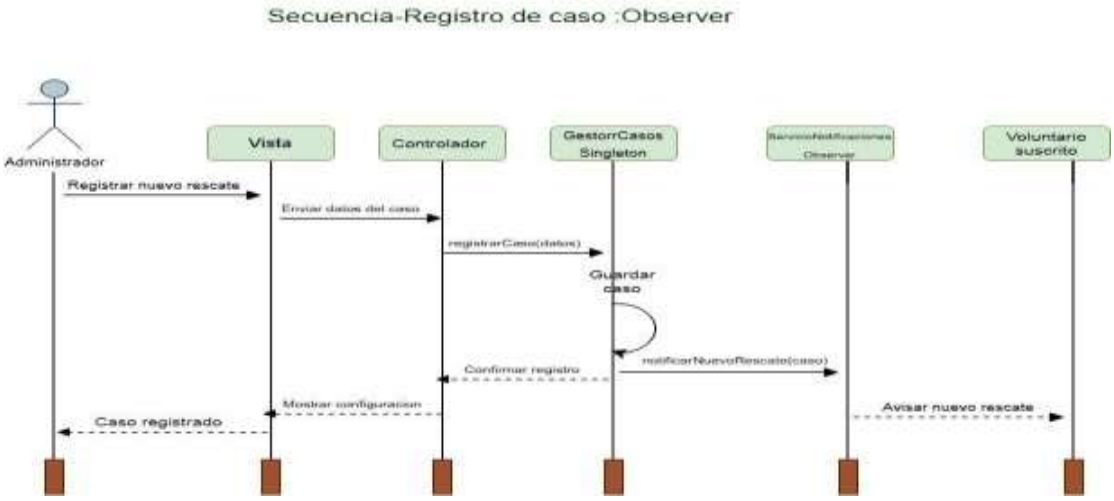


Fig.7: **Sistema de Alertas Automáticas Patrón Observer e Integración Singleto**

El actor envía la solicitud al motor único GestorCasos (Singleton). Tras asegurar la instancia y procesar el ingreso, este delega el evento al ServicioNotificaciones. Dicha clase ejecuta el método de notificación masiva, invocando de forma síncrona la operación +actualizar () en la línea de vida del Voluntario sin acoplar directamente sus componentes.

5.3. Diagrama de Secuencia: Creación Dinámica de Expedientes (Patrón Factory Method).

Modela el flujo temporal en el que el sistema de fábrica de forma automatizada el expediente de un animal según su especie.

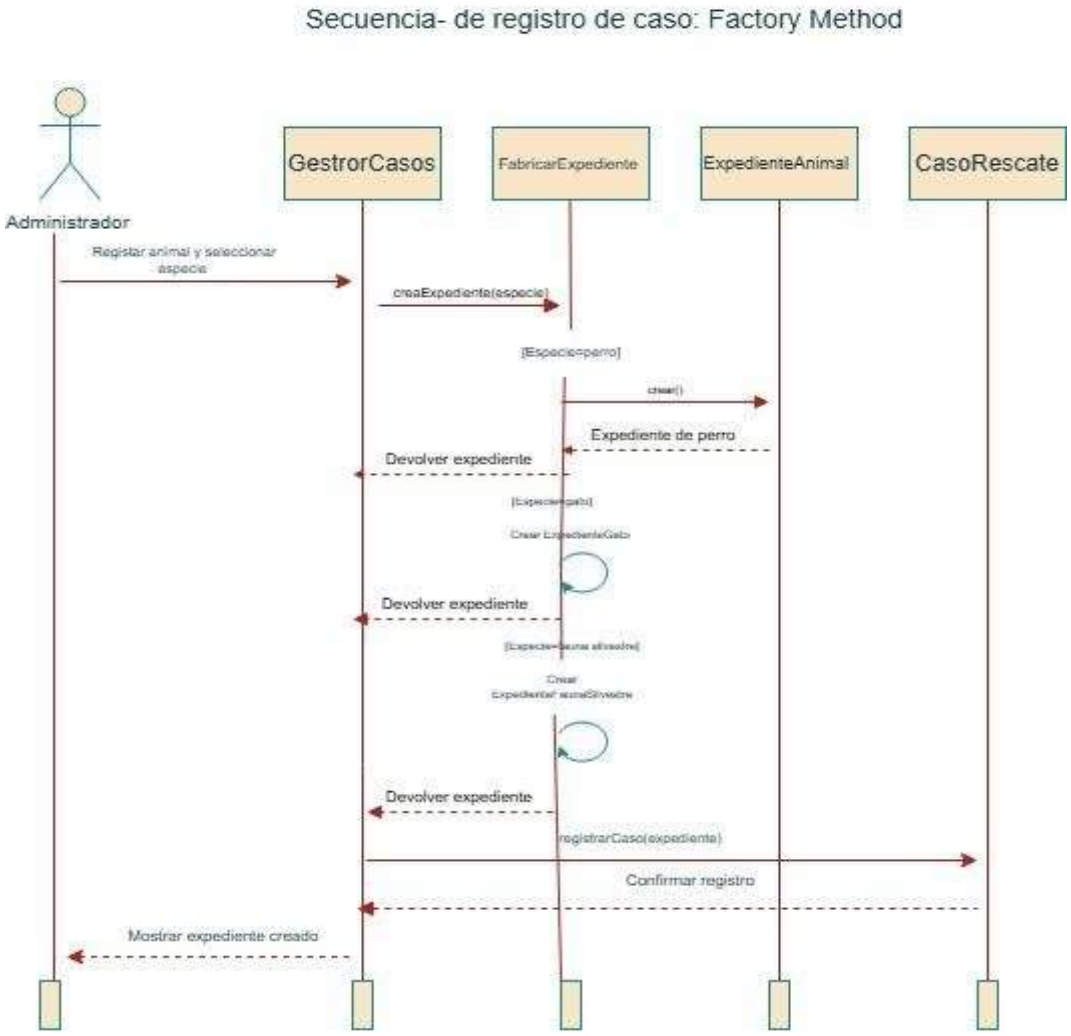


Fig.8: Creación Dinámica de Expedientes (Patrón Factory Method).

El motor central GestorCasos invoca la operación +crearExpediente() en la clase FabricaExpediente. El sistema evalúa condicionalmente la especie ingresada e instancía dinámicamente el objeto específico (Perro, Gato o Fauna Silvestre) bajo la interfaz común Animal. Tras recibir el objeto creado, el gestor delega la persistencia de los datos a la entidad CasoRescate

Arquitectura General del Sistema: Flujo de Versiones (Gitflow)

Se detalla la transición controlada desde la fase inicial en la rama de análisis hasta el despliegue final e inmutable del informe PDF en la rama Maestro.

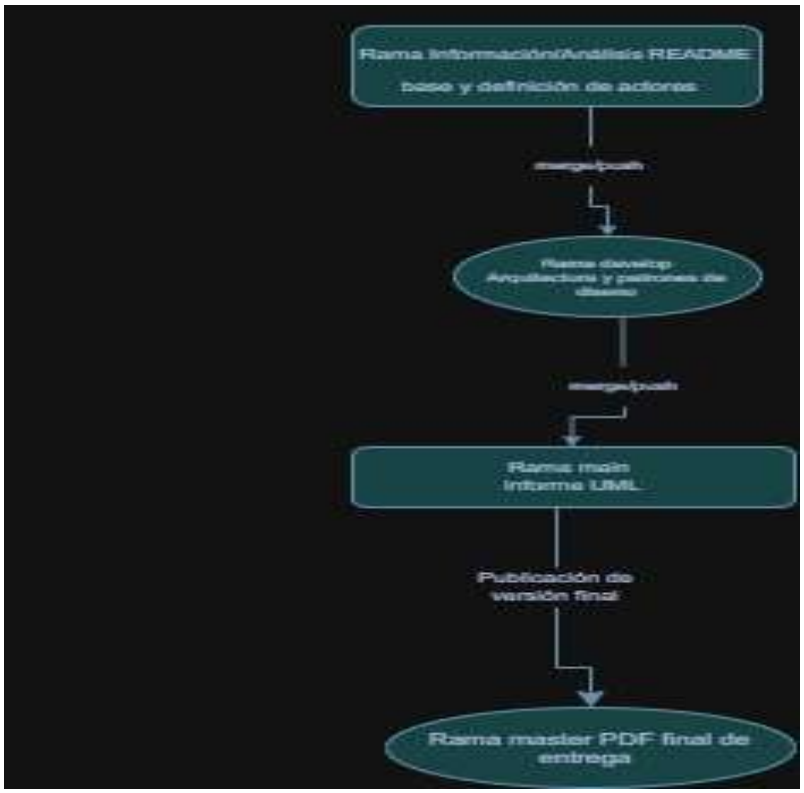


Figura 3: *Diagrama visual de la arquitectura general del sistema integrando el flujo de versiones (Gitflow)*

6.Evidencias Proceso Control de Versiones y Gitflow

Enlace (GitHub):

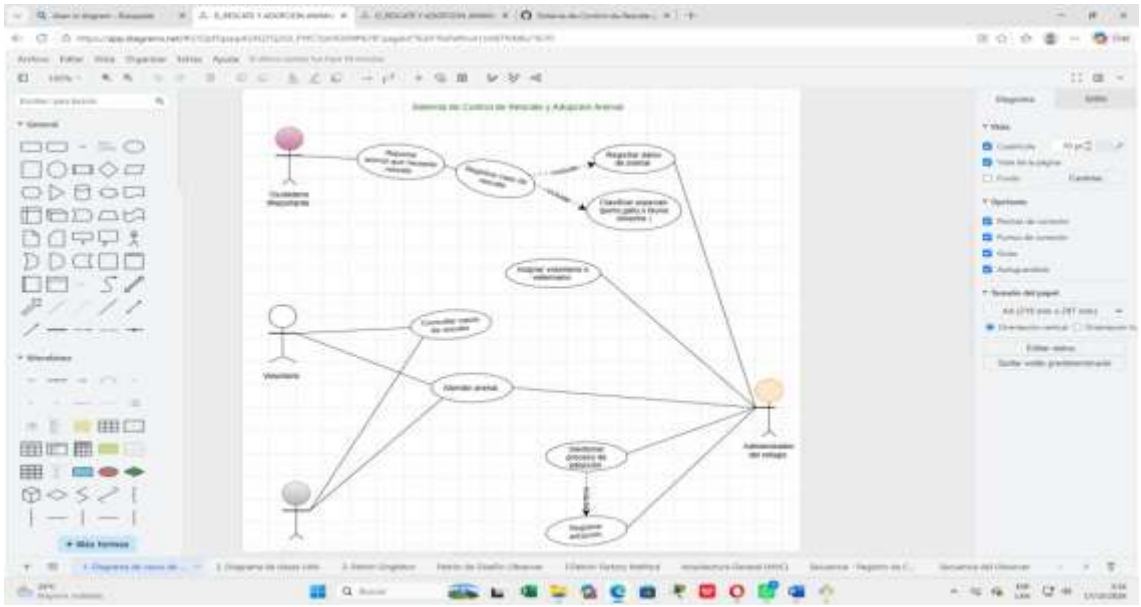
https://github.com/greyna3538-hub/Sistema-de-Control-de-Rescate-y-Adopcion-Animal.git

Enlace (Draw.io):

https://drive.google.com/file/d/1QzfTqzaqiA2XQ7Q2Gt_PYfC7jstrR309/view?usp=drive_link

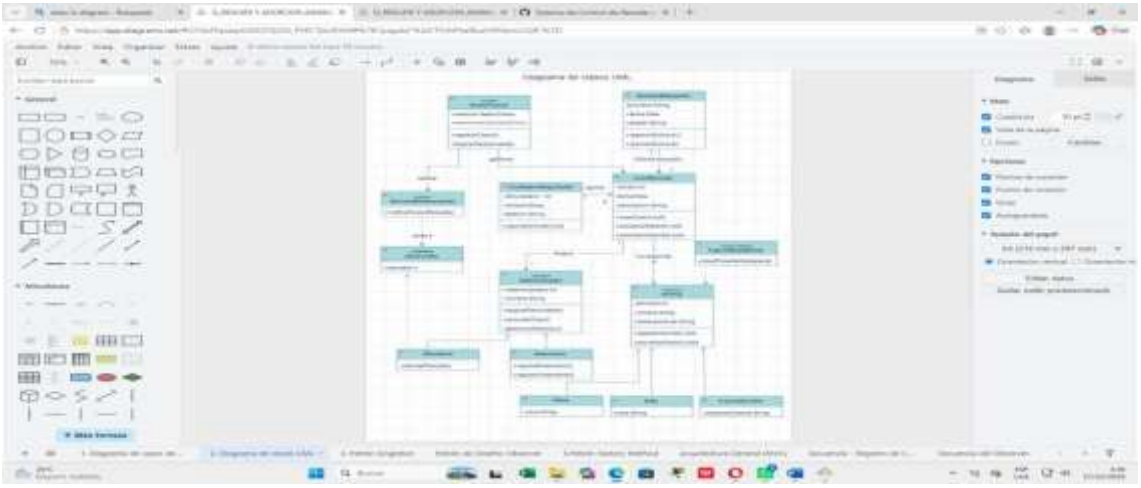
6.1. Capturas de Pantalla Draw.io:

Diagrama de casos de uso



Captura.1: Diagrama de casos de uso

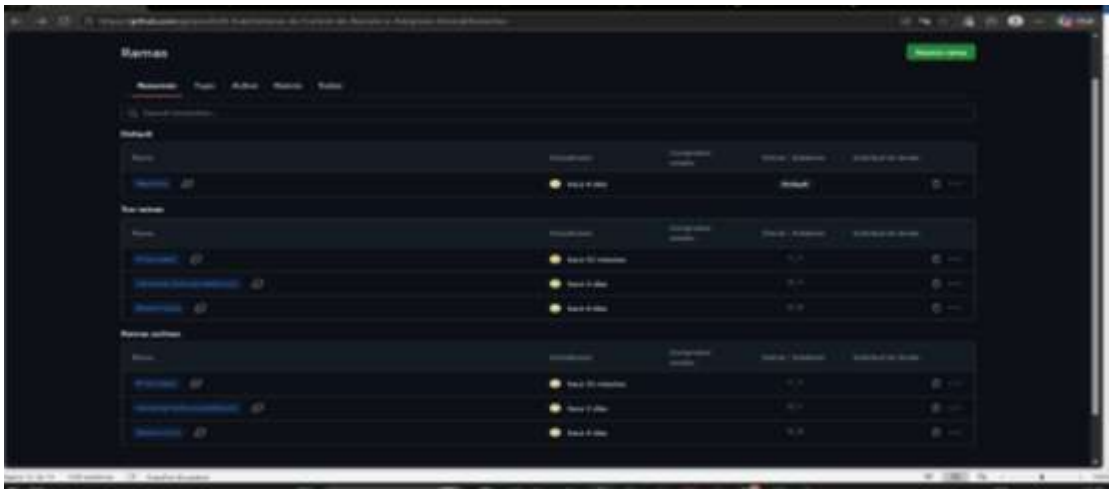
Diagrama de clases UML



Captura.2: Diagrama de clases UML

6.2. Capturas de Pantalla GitHub:

Estructura Base de Ramas

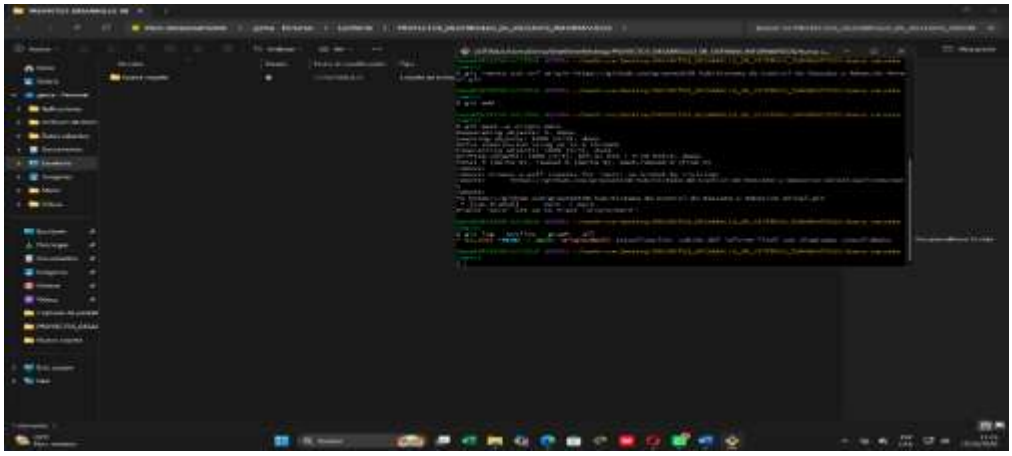


Captura 4: Estructura de ramas del repositorio GitHub.

Para la gestión del Sistema de Control de Rescate y Adopción Animal, se implementó el flujo de trabajo profesional Gitflow, garantizando el orden y la trazabilidad del diseño de software. Se utilizó la rama Maestro (master) como el entorno definitivo para la entrega del informe consolidado. Asimismo, se crearon de forma ordenada las ramas Principal (main) y Desarrollo (develop) para la integración progresiva de los componentes, y la rama Característica/análisis (feature/analisis) para el modelado inicial de los requisitos y actores del refugio, cumpliendo con los estándares profesionales exigidos en la actividad

Capturas Código de Ejecución:

Se evidencia la consolidación absoluta de la arquitectura Gitflow, mostrando la perfecta sincronización de las ramas locales con los punteros remotos del servidor de GitHub



Captura 5: Captura de pantalla de la terminal de Git Bash de la aplicación local

.

Se evidencia la publicación exitosa

Archivo Actividad_1_Gema_Reyna_D.S.I..pdf dentro de la rama raíz Maestro (master), confirmando la disponibilidad inmediata de la documentación arquitectónica para la evaluación del docente.

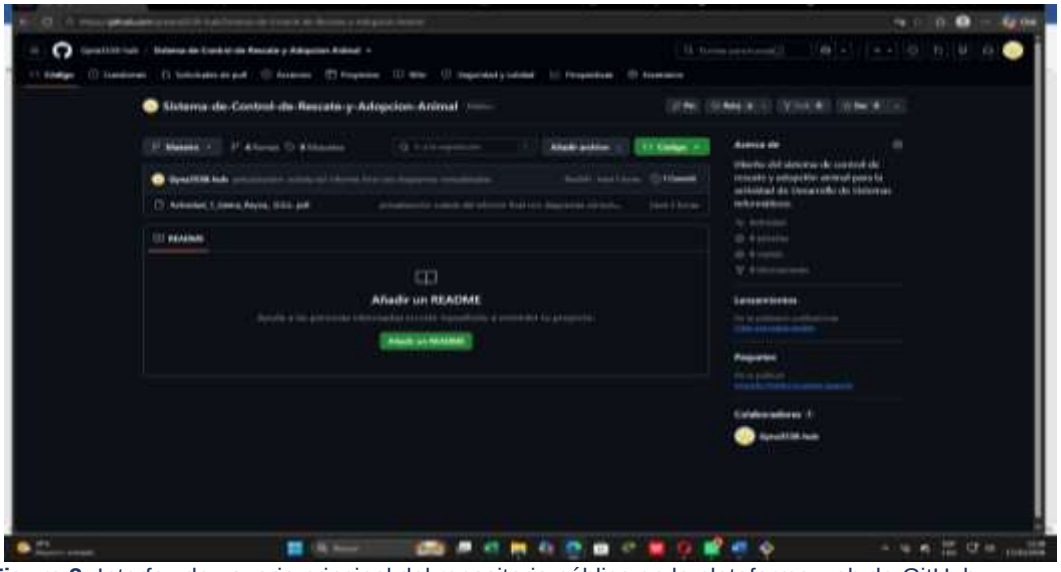


Figura 3: Interfaz de usuario principal del repositorio público en la plataforma web de GitHub

Rama de desarrollo de características (*feature/análisis*)

Se evidencia el aislamiento correcto de la fase de análisis inicial del sistema, conteniendo únicamente la documentación base del proyecto en un archivo README.md, lo que demuestra un uso riguroso y profesional de la metodología Gitflow sin mezclar el entregable final en ramas temporales.

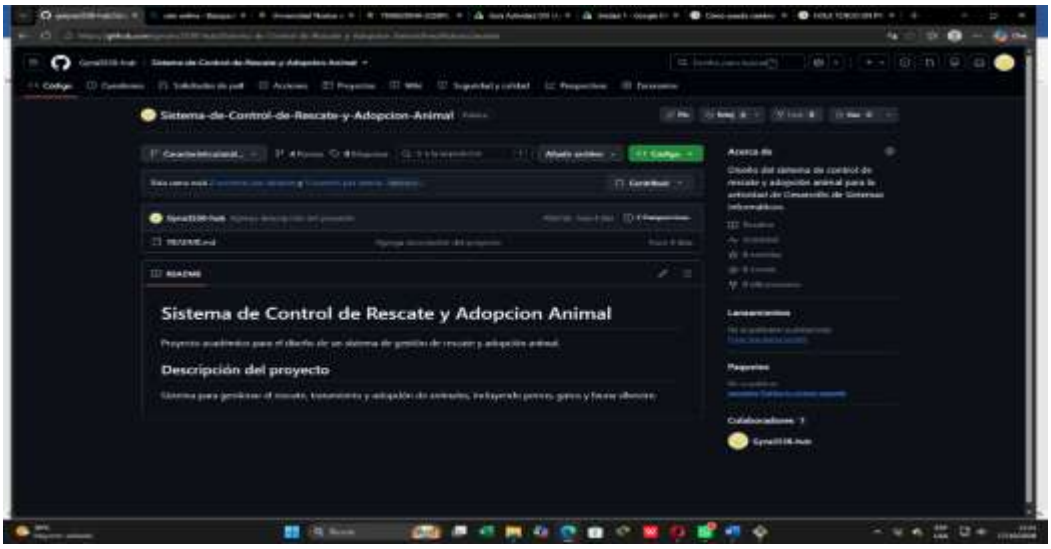


Figura 4: *Vista de la rama de desarrollo de características (feature/analisis) en GitHub.*

Rama de integración Desarrollo (develop)

Se evidencia la correcta inicialización del entorno base del proyecto a través de un archivo de marcado README.md, demostrando el cumplimiento estricto del modelo Gitflow al mantener un espacio de trabajo limpio y aislado para la futura unificación de los componentes lógicos del sistema.

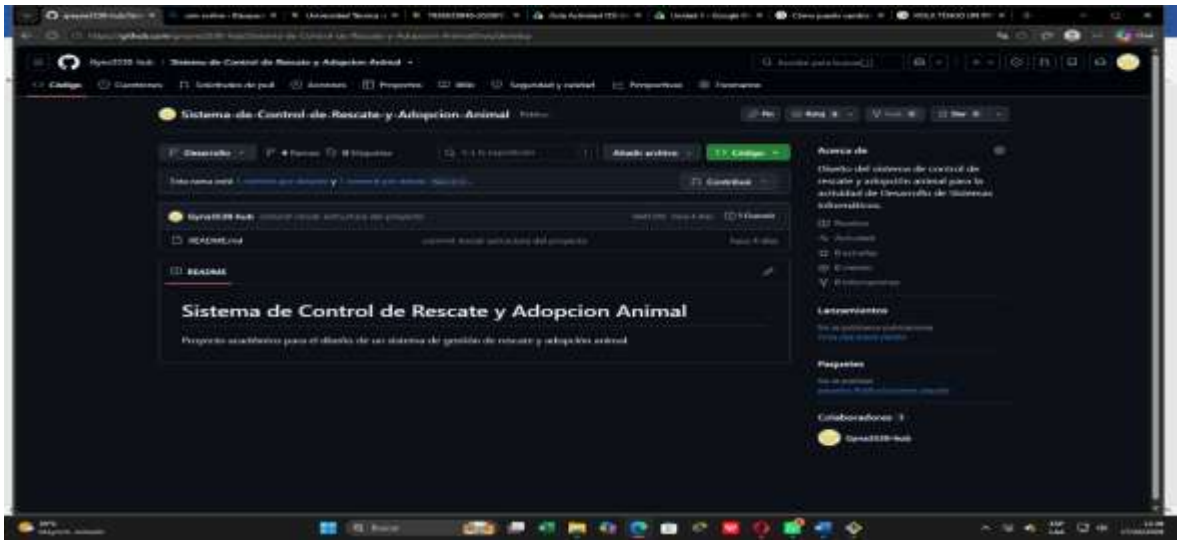


Figura 5: *Vista de la rama de integración **Desarrollo (develop)** en la plataforma web de GitHub.*

Bibliografía

GitHub. (s. f.). GitHub Docs. https://docs.github.com/

draw.io. (s. f.). UML diagrams. https://www.drawio.com/docs/diagram-types/uml/

draw.io

Object Management Group. (2017). Unified Modeling Language (UML), version 2.5.1. https://www.omg.org/spec/UML/2.5.1/About-UML OMG

Refactoring.Guru. (s. f.). Design patterns. https://refactoring.guru/design-patterns/catalog Refactoring Guru

draw.io. (s. f.). Drawing Gitflow diagrams in draw.io. https://www.drawio.com/docs/diagramtypes/gitflow-diagram/